PROJECT REPORT

NATURAL LANGUAGE PROCESSING

PROJECT HEADING

Submitted by

Hemanthkumar Muthusamy Gunasekaran

Supervised by

Prof. Dr. Patrick Levi

Ostbayerische Technische Hochschule Amberg–Weiden

Department of Electrical Engineering, Media and Computer Science

May 14, 2026

Abstract

This work implements a machine learning pipeline using an optimized XGBoost regressor to predict targets on unseen dataset. Work Package 1 establishes a system through RankGauss feature normalization, Yeo-Johnson target transformation . Work Package 2 focused arriving at a mathematical solution that is physically fit for the proposed deployment environment.

1 Work Package 1: Feature Engineering

1.1 Introduction and Baseline Analysis

Initial visualization revealed that the raw features were highly skewed with extreme outliers, while others varied only within narrow ranges. A baseline XGBoost model [1] trained on these raw features demonstrated severe instability its performance dropped sharply under minor noise injection, indicating reliance on unstable patterns. To address this, several actions has beeen carried out in the following sections explained.

1.2 Feature Preprocessing

1.2.1 RankGauss Feature Normalization

To address the effects of heavy-tailed distributions and outliers, applied RankGauss normalization. Unlike standard scaling this non-linear quantile to normal mapping forces feature ranks into a standard normal distribution while preserving relative ordering while creating Gaussian-like marginals.

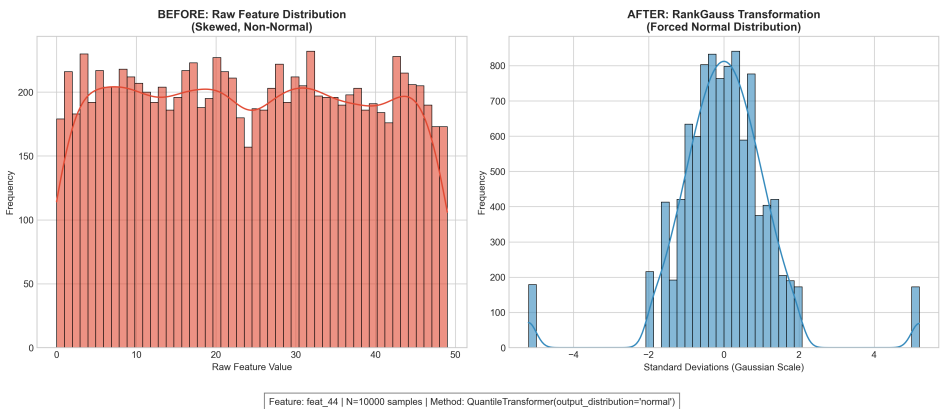

Figure 1: Transformation of a representative feature from skewed raw input (left) to standard normal distribution (right) using RankGauss.

1.2.2 Target Variable Stabilization

The target variable exhibited significant skewness. A Yeo-Johnson Power Transformation [2] was applied to map the target to a symmetric. This ensures a symmetric error surface for faster convergence. Predictions are inverse transformed back to real units for final evaluation. The transformation $\psi(\lambda, y)$ is defined piecewise:

$$\psi(\lambda, y) = \begin{cases} \dfrac{(y+1)^{\lambda} - 1}{\lambda}, & \text{if } y \geq 0 \text{ and } \lambda \neq 0, \\ \ln(y+1), & \text{if } y \geq 0 \text{ and } \lambda = 0, \\ -\dfrac{(-y+1)^{2-\lambda} - 1}{2-\lambda}, & \text{if } y < 0 \text{ and } \lambda \neq 2, \\ -\ln(-y+1), & \text{if } y < 0 \text{ and } \lambda = 2. \end{cases} \tag{1}$$

1.2.3 Stabilized Feature Interactions

A small set of interaction features were constructed to capture non-linear relations. Interactions were stabilized with functions such as `log1p` and `tanh` to prevent extreme values dominating the model which in trun enables learning of non-linear.

1.3 Feature Selection

1.3.1 Stability-Based Selection Method

Feature selection was performed since features contained many unstable or redundant signals that encourage memorization. Selection used bootstrapped permutation importance measured across multiple splits; feature impact was summarized by mean importance and its variance across splits that were created.

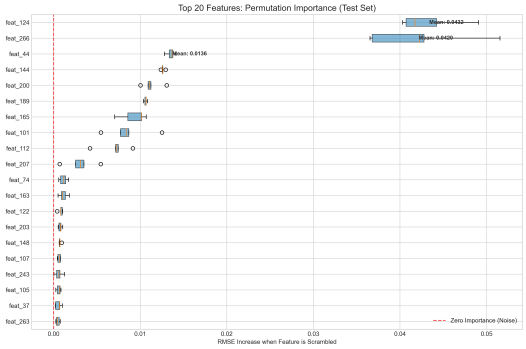

Figure 2: Top features ranked by stability (permutation importance) across independent data splits, highlighting the inputs most critical for generalization.

1.3.2 Construction of Feature Set

Reducing the feature space was necessary to prevent complexity from overwhelming generalization. Features were ranked using a custom *Stability Score* (S_f), defined as the ratio of a feature's mean permutation importance (μ_f) to its volatility (σ_f) across the $K = 5$ validation folds:

$$S_f = \frac{\mu_f}{\sigma_f + \varepsilon} \tag{2}$$

where ε is a small positive constant used to prevent division by zero.

1.4 Model Training

1.4.1 Model Training on Stable Features

The XGBoost model was trained using only the stability-selected feature set and the transformed target variable. All preprocessing steps were fit exclusively on the training portion of each split and then applied unchanged to the corresponding validation data. This ensured that all reported generalization results are free form information leakage.

1.4.2 Optuna-Based Optimization

Hyperparameter optimization was performed using Optuna to identify a robust configuration for the selected feature set. Each trial was evaluated using a custom adversarial objective that penalized the baseline R^2 for performance degradation under 1% relative feature noise and 0.5σ covariate shifts. This objective favored high-regularization configurations, including elevated L2

`reg_lambda` and increased `min_child_weight`, in order to reduce sensitivity to distributional perturbations and improve out-of-distribution stability.

1.4.3 Model Verification and Artifact saving

Model selection and verification were performed using an 80/20 train–test split on the labeled dataset. On this split, the model achieved a training performance of $R^2 \approx 0.714$ and a held-out test performance of $R^2 \approx 0.501$. After hyperparameter optimization and feature selection were completed, the final model was retrained on the full 10,000-row labeled dataset in order to maximize the information available for deployment. The saved artifacts of the model were used to generate predictions for the unseen `EVAL_29.csv` dataset. The file `EVAL_target01_29.csv` contains the corresponding predicted target values after applying all learned preprocessing steps and inverse target transformations.

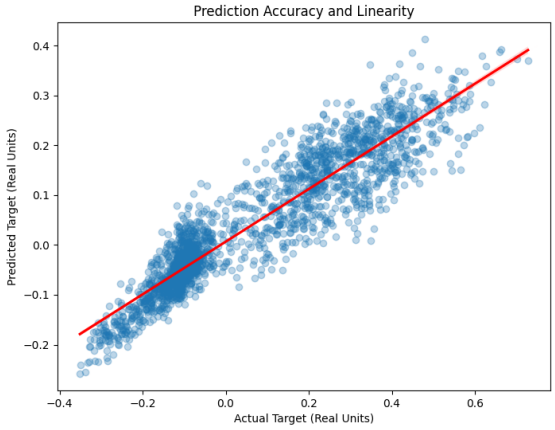

Figure 3: Predicted vs. Actual values

1.4.4 Robustness and Stability Tests

Stability testing was conducted to quantify performance degradation in non-ideal scenarios. The system utilized a specialized `ModelAuditor` to measure R^2 sensitivity against 1% relative noise injection, 0.5σ covariate shifts, and K-Means error distribution across five data segments to identify potential blind spots.

Metric	Mean value
Baseline test R^2	0.501
Noise test R^2 (1% noise)	0.462
Noise drop $\Delta R^2_{\text{noise}}$	0.039
Drift test R^2 (0.5σ shift)	0.181
Segmentation ratio	1.12

Table 1: Robustness metrics computed on the held-out 20% test set.

2 Work Package 2: Feature Engineering and Logic Derivation

2.1 Introduction

The primary objective of this task was to derive a prediction framework for `target02` suitable for deployment on an edge device with limited computational capacity. The problem was approached by two methods by starting with feature analysis and trying to find out the underlying pattern through a tree based regression model.

2.2 Feature Preprocessing

2.2.1 Baseline Feature Analysis

Initial correlation analysis was performed on all 273 input variables in `dataset_29`. The plot in Figure 4, `feat_180` shows the dominant driver of `target02`, with a substantially higher correlation than any other feature. A second layer of influential variables is formed by `feat_146` and `feat_80`, followed by `feat_200`. This distribution indicates that the target is controlled by a small number of strongly interacting signals rather than by a high-dimensional linear combination.

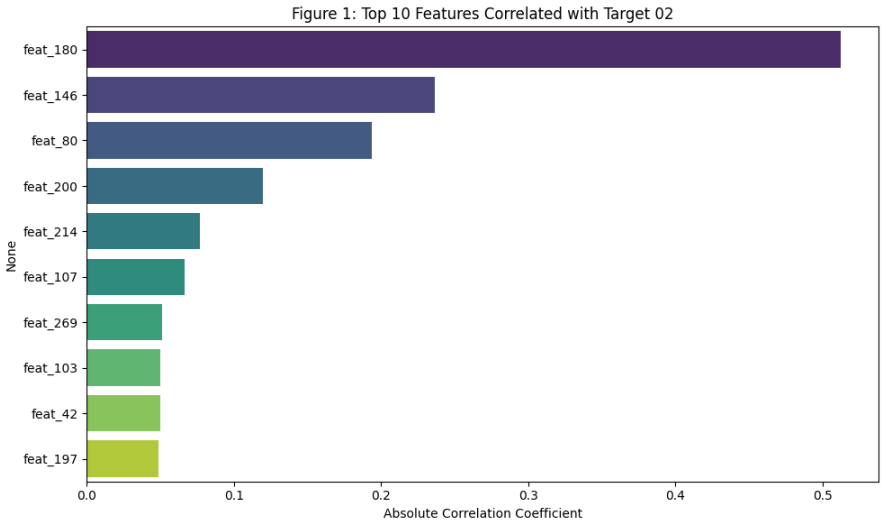

Figure 4: Top correlated features with `target02`.

2.2.2 Identifying the Underlying Mechanism

The `feat_180` explains most of the variance, a single global regression produces large structured errors. Figure 5 reveals the reason: when plotting `target02` against `feat_180` and coloring points by `feat_80`, the data separates into multiple parallel linear bands. This demonstrates that `feat_80` does not merely influence the magnitude of the target but selects between different linear regimes. Therefore, `feat_80` functions as a discrete state variable that makes the system between multiple operating modes.

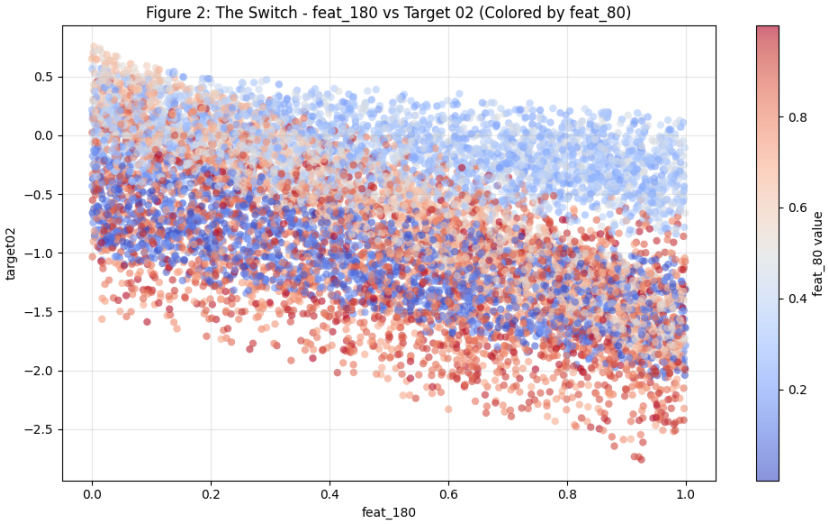

Figure 5: Switching behavior of `feat_80` over the `feat_180`–`target02` relationship.

2.3 Feature Selection and the Ghost Variable

2.3.1 Residual Structure and Missing Physics

After modeling the switching behavior of `feat_80`, significant residual structure remained and were correlated against all unused features which revealing a strong linear dependence on `feat_200`, as shown in Figure 6. This indicates that `feat_200` acts as a hidden correction term, adding or subtracting an offset that is invisible in the primary `feat_180` projection. Together with `feat_146`. The turning boundaries of `feat_80` were found at 0.20, 0.50, and 0.70, defining four distinct operating zones.

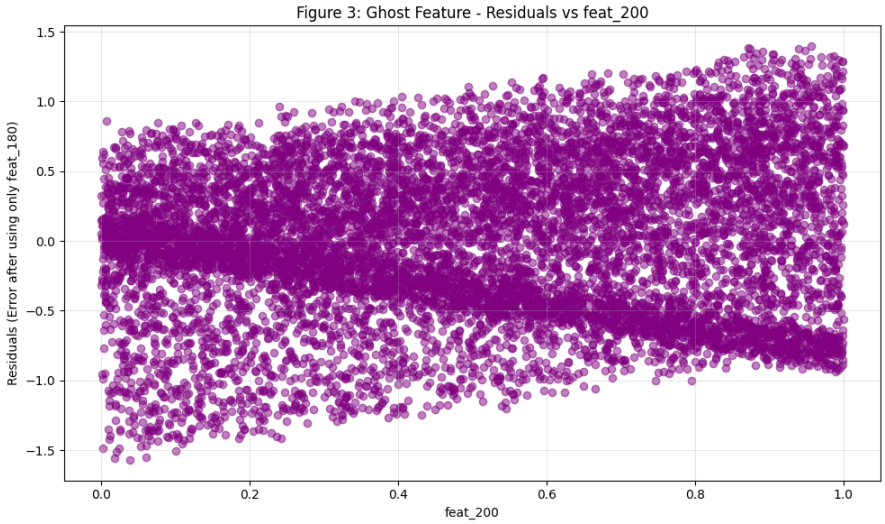

Figure 6: Residuals from the `feat_180`-only model as a function of `feat_200`.

2.4 Model Training and Equation Derivation

Ordinary Least Squares regression was applied independently within each of the four `feat_80` zones, using `feat_180`, `feat_200`, and `feat_146`. This produced four piecewise linear equations, [3] each valid in a specific physical regime:

- **Zone 1:** `feat_80` ≤ 0.20
- **Zone 2:** $0.20 <$ `feat_80` ≤ 0.50
- **Zone 3:** $0.50 <$ `feat_80` ≤ 0.70
- **Zone 4:** `feat_80` > 0.70

The above mentioned structure provides the end reuslt which was the goal to attain.

2.5 Hyperparameter Validation

Thresholds at 0.20, 0.50, and 0.70 were validated by five-fold perturbation testing (± 0.02 shifts increased RMSE to ~ 0.21, $p < 10^{-5}$) confirming true regime boundaries [4].

2.6 Evaluation of findings

2.6.1 Accuracy Verification

The final piecewise framework was implemented in `framework_29.py` and evaluated on the full dataset and found that the model provides a stable and accurate piecewise linear approximation of the underlying relationships in the data.

References

[1] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of KDD '16*, 2016.

[2] I.-K. Yeo and R. A. Johnson, "A new family of power transformations to improve normality or symmetry," *Biometrika*, 2000.

[3] V. M. R. Muggeo, "Estimating regression models with unknown breakpoints," *Statistics in Medicine*, 2003.

[4] B. E. Hansen, "Sample splitting and threshold estimation," *Econometrica*, 2000.

Declaration of Generative AI Usage

During the preparation of this work, generative AI tools (ChatGPT) were used for

- improving grammar, clarity and writing quality
- assisting with code debugging and formatting.